PROJECT TITILE

Linux Task Automation Platform

S Nikhil

CYBER SECURITY

TAH0901261959

TEKS ACADEMY

8309682923

SHASHIDHAR

CYBER SECURITY MENTOR

TEKS ACADEMY

1.Abstract :

In modern IT environments, system administrators are responsible for performing many routine tasks such as system monitoring, file backups, temporary file cleanup, and system maintenance. Performing these tasks manually can be time-consuming and may lead to human errors or delays in system maintenance.

The **Linux Task Automation Platform** is designed to simplify and automate these repetitive administrative tasks in Linux environments. This project uses shell scripting and Linux automation tools to automatically perform important system operations without constant manual intervention.

The platform provides automation for tasks such as system backups, disk usage monitoring, cleaning temporary files, and running scheduled system operations. These tasks are executed using **Bash scripts** and scheduled with the **Cron job scheduler**, which allows the system to run scripts automatically at predefined times.

Additionally, the platform maintains **log files** that record the status of executed tasks, making it easier for administrators to monitor system activity and troubleshoot issues if they occur.

By implementing this automation platform, system administrators can reduce manual workload, improve system reliability, and ensure that routine maintenance tasks are performed consistently and efficiently.

Overall, this project demonstrates how Linux automation tools can be used to improve system management, enhance productivity, and provide a more organized approach to server administration.

2. Problem Statement

In today's computing environment, servers and systems require constant monitoring and maintenance to ensure smooth operation. System administrators often need to perform repetitive tasks such as backing up files, monitoring disk usage, cleaning temporary files, and running maintenance scripts.

However, performing these tasks manually can create several problems. Administrators may forget to perform certain tasks, delay maintenance activities, or make mistakes while executing commands. This can result in system inefficiencies, storage problems, and potential system failures.

When tasks such as disk monitoring or backups are not performed regularly, systems may experience performance issues or even data loss. Additionally, managing these tasks across multiple systems can become a difficult and time-consuming process.

For example, if temporary files are not cleaned regularly, they can consume large amounts of disk space and slow down the system. Similarly, if backups are not created frequently, important data may be lost during system failures.

Therefore, there is a need for a system that can automatically perform these repetitive administrative tasks without requiring constant manual intervention.

The solution is to develop an **automated platform that schedules and executes system tasks automatically using Linux tools such as Bash scripting and Cron scheduling.** This automation helps administrators manage systems more efficiently and ensures that important maintenance tasks are performed regularly.

3.Project Objectives

The main objective of the **Linux Task Automation Platform** is to automate routine system administration tasks in a Linux environment.

Automate Routine System Tasks

The primary goal of this project is to automate common administrative tasks such as file backups, disk monitoring, and temporary file cleanup. This reduces the need for manual execution and ensures tasks are performed consistently.

Improve System Efficiency

By automating maintenance tasks, the system can operate more efficiently. Automated scripts help maintain system performance by regularly cleaning unnecessary files and monitoring system resources.

Reduce Manual Workload

System administrators often spend significant time performing repetitive operations. Automation allows administrators to focus on more important tasks instead of repeatedly performing routine maintenance.

Implement Scheduled Task Execution

The project uses the **Cron scheduler** to automatically run scripts at scheduled times. This ensures that system tasks are executed regularly without requiring human intervention.

Maintain Task Logs

The platform records task execution details in log files. These logs help administrators track system activities and identify potential issues quickly.

Provide a Simple Automation Platform

The system is designed to be simple and easy to manage so that even users with basic Linux knowledge can configure and run the automation platform.

4. Project Scope

This project focuses on automating common system administration tasks in a Linux environment. The **Linux Task Automation Platform** uses Bash scripting and Cron scheduling to perform tasks automatically, reducing manual effort and improving system management.

Key features of the project include:

• **Automated Task Execution:** The system automatically performs tasks such as file backup, disk monitoring, and temporary file cleanup using scheduled scripts.

• **Scheduled Automation:** Tasks are executed at predefined times using **Cron jobs**, ensuring regular system maintenance.

• **Logging System:** The platform records the execution of tasks in log files to help administrators monitor system activity.

• **Linux-Based Implementation:** The solution is designed to run on Linux operating systems such as **Ubuntu** using built-in Linux tools.

Limitations:

• The project is designed only for **Linux systems**.
• It mainly works through **command-line scripts** and basic web interface support.

5. Tools and Technologies Used

To develop the **Linux Task Automation Platform**, several Linux tools and technologies are used to automate and manage system tasks.

Ubuntu Linux

Ubuntu Linux serves as the primary operating system for this project. It provides a stable environment for running scripts and automation tools.

Bash (Shell Scripting)

Bash scripting is used to create automation scripts that perform tasks such as file backups, disk monitoring, and system cleanup.

Cron Scheduler

Cron is a built-in Linux scheduler that allows scripts to run automatically at predefined intervals. It is responsible for executing automation scripts without manual intervention.

Apache Web Server (Optional Dashboard)

Apache can be used to create a simple web interface where administrators can manually trigger automation scripts through a browser.

PHP

PHP can be used to connect the web interface with system scripts and execute automation commands.

Text Editors (Nano / Vim)

Text editors like Nano or Vim are used to write and edit the automation scripts.

VMware Workstation

VMware Workstation is used to create a virtual Linux environment for testing and running the project safely without affecting the host system.

6. Deliverables

At the completion of the project, several deliverables will be produced to demonstrate the successful implementation of the Linux Task Automation Platform.

Automation Scripts

A set of Bash scripts will be developed to automate tasks such as file backups, disk usage monitoring, and temporary file cleanup.

Scheduled Task Configuration

Cron job configurations will be created to schedule the execution of automation scripts at specific times.

Log Management System

Log files will be generated to record the results of executed scripts and maintain a history of system tasks.

Project Documentation

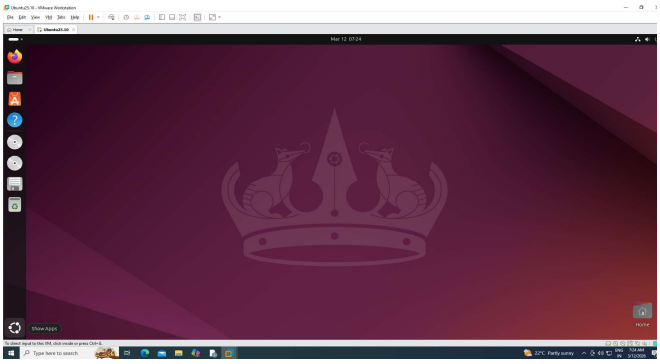
Complete documentation explaining the project objectives, implementation process, tools used, and system functionality will be provided.

Demonstration Setup

A working demonstration of the Linux Task Automation Platform will be prepared using a virtual machine environment.

7.Project Implementation

Step 1: Install Ubuntu in VMware



Step 2: Install Apache2 on Ubuntu

```
student@UBUNTU-VM:~$ sudo apt update
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
9 packages can be upgraded. Run 'apt list --upgradable' to see them.
student@UBUNTU-VM:~$ sudo apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.58-1ubuntu8.11).
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
student@UBUNTU-VM:~$ sudo systemctl start apache2
student@UBUNTU-VM:~$ sudo systemctl enable apache2
```

Step 3: Test Apache Web Server

Open browser inside VM and type:

```
http://localhost
```

Step 4: Create Web Interface for Automation Platform

```
student@UBUNTU-VM:~$ cd /var/www/html
student@UBUNTU-VM:/var/www/html$ sudo mkdir automation
mkdir: cannot create directory 'automation': File exists
student@UBUNTU-VM:/var/www/html$ cd automation
```

Step 5: Simple Web Page to Run Scripts

Create file:

```
student@UBUNTU-VM:/var/www/html/automation$ sudo nano index.php
```

Code:

```
<?php
echo "<h1>Linux Task Automation Platform</h1>";

if(isset($_GET['backup'])){
    system("bash /home/user/linux-automation-project/scripts/backup.sh")
    echo "Backup Completed";
}

if(isset($_GET['monitor'])){
    system("bash /home/user/linux-automation-project/scripts/system_moni
    echo "Monitoring Done";
}

if(isset($_GET['cleanup'])){
    system("bash /home/user/linux-automation-project/scripts/cleanup.sh"
    echo "Cleanup Completed";
}
?>

<a href="?backup=1">Run Backup</a><br>
<a href="?monitor=1">Run Monitoring</a><br>
<a href="?cleanup=1">Run Cleanup</a>
```

Step 6: Install PHP (Needed for Apache)

```
student@UBUNTU-VM:/var/www/html/automation$ sudo apt install php libapache2-mod-php
```

Start apache2 server

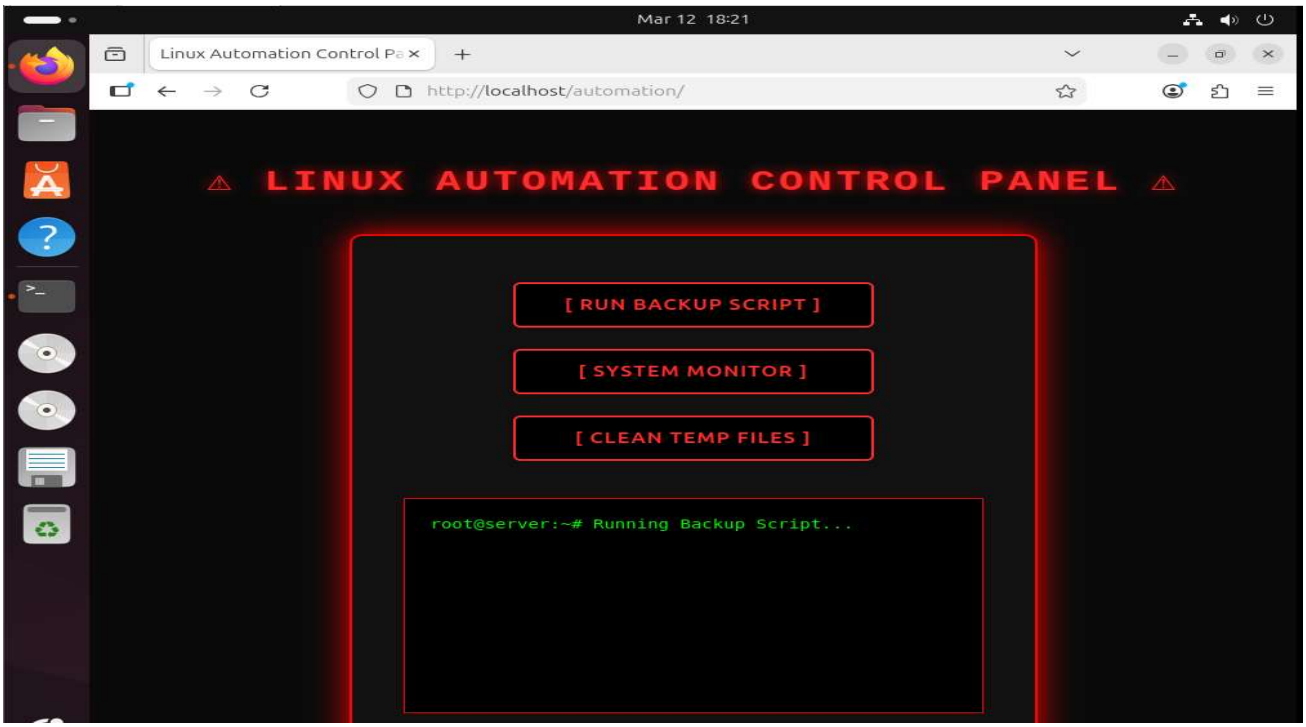
```
student@UBUNTU-VM:/var/www/html/automation$ sudo systemctl start  apache2
```

Step 7 : Open Your Automation Platform

Browser:

```
http://localhost/automation
```

Output:



(Final Project Architecture):

```
VMware
   ↓
Ubuntu Linux
   ↓
Apache2 Web Server
   ↓
PHP Web Interface
   ↓
Bash Automation Scripts
   ↓
Cron Scheduler
```

8.Conclusion:

The **Linux Task Automation Platform** successfully demonstrates how system administration tasks can be automated using Linux tools and scripting. By integrating **Bash scripts, Cron scheduling, and a web-based interface**, the system provides a simple and efficient way to manage repetitive tasks.

This project helps reduce manual workload, improve system efficiency, and enhance administrative productivity. It also provides practical experience with **Linux administration, automation, and web integration**, which are essential skills in the fields of **system administration and cybersecurity**.

The platform can be further enhanced by adding features such as **remote server automation, advanced monitoring tools, email alerts, and user authentication for secure access.**